

Introduction to RNNs

Motivating Example: Negation in Sentiment Analysis

- Recall: one practice to overcome the weakness of bag-of-words models is to add NOT_ to every word between negation and following punctuation:

didn't like this movie , but I



didn't NOT_like NOT_this NOT_movie but I

Negation in Sentiment Analysis

- This is a very crude solution:
 - Explicit negation is only one way to reverse polarity
 - “He **failed** to convey the importance of his message”
 - Negation scope:
 - “I didn’t like the exposition in this otherwise excellent film”
 - Logical structure can be more complex
 - “I don’t think anyone could have done a better job”
- More recent approaches take the context (surrounding words) into account

Context Length

- We've seen that Markov models (for language modeling, prediction in many cases etc.) are effective, but do not capture dependencies beyond a certain length
- By the Markov assumption:
 - It's enough to know the last n tokens you've encountered to know what's next
 - Alternative view: the probability of a sequence is the product of the probabilities of its *n-grams*

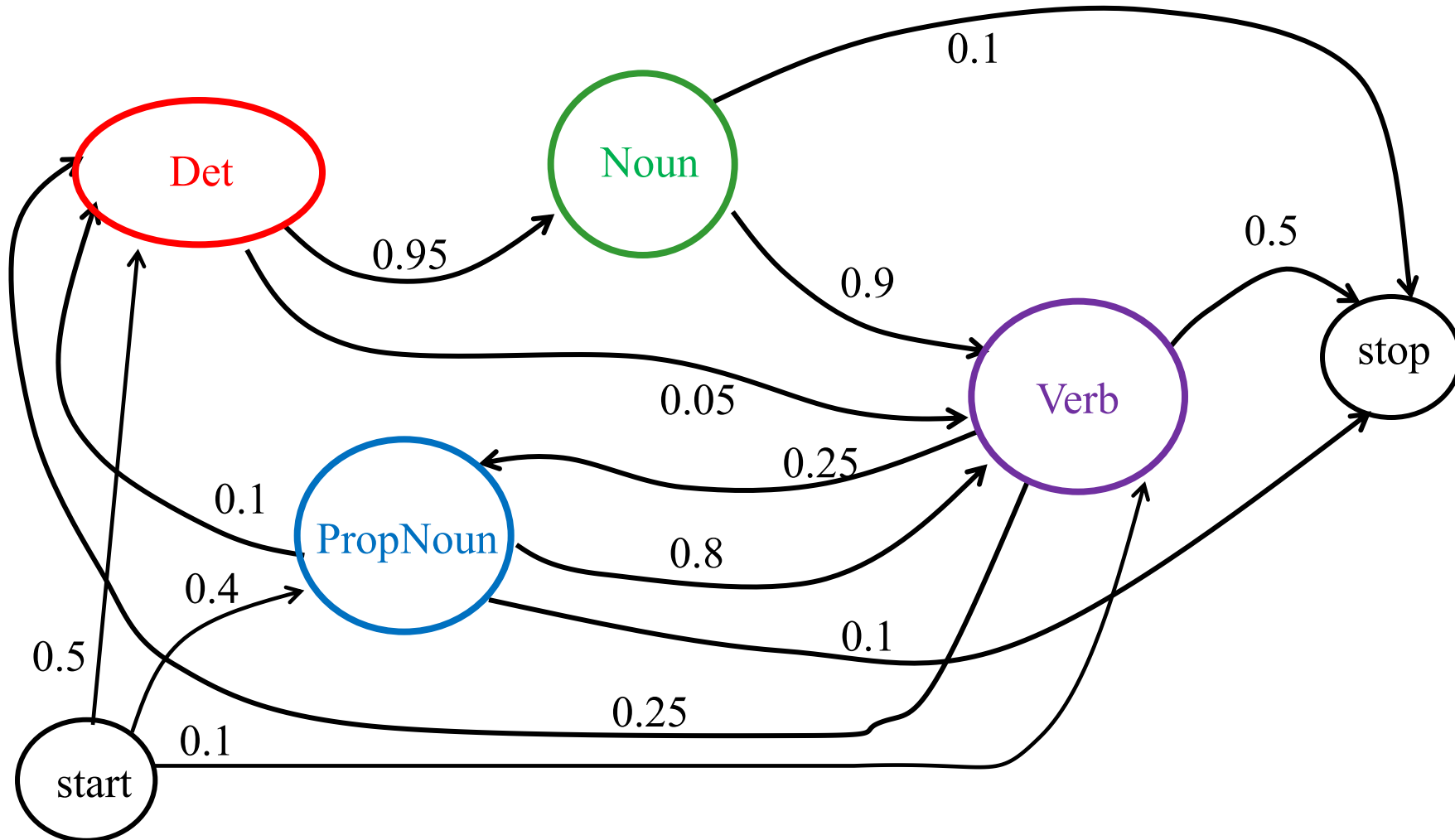
The brown cat that disturbed my sleep yesterday _____ (is/are)

The brown cats that disturbed my sleep yesterday _____ (is/are)

Sentiment Analysis as Sequence Labeling

- Consider the example:
 - *How can you not see this movie?*
 - *You should not see this movie*
- How well will a bi-gram model work?
 - Similar unigrams and bigrams → similar prediction
- The problem with Markov Models: need to maintain a **state** to capture distant influences
 - The size of the space increases exponentially with the order of the model

Recall: Markov Models are Finite State Automata (FSAs) with transition probabilities



Recurrent Neural Networks

- Motivation:
 - Neural network model, but with a state
 - How can we borrow ideas from FSAs?
- RNNs are FSAs ...
 - With a twist
 - No longer finite in the same sense
 - The state is an embedding of the history in a continuous space
 - The embedding function of the history to a vector is learned as well

Recurrent Neural Networks

- Map from dense sequence to dense representation
 - Maps a sequence of vector inputs to a sequence of vector states

$$x_1, \dots, x_n \rightarrow s_1, \dots, s_n$$

- A (parametrized) transition function R does the mapping:

$$s_i = R(s_{i-1}, x_i)$$

- R is parametrized and parameters are shared across steps

$$s_4 = R(s_3, x_4) = R(R(s_2, x_3), x_4) = R(R(R(s_1, x_2), x_3), x_4)$$

Recurrent Neural Networks

- An output function O maps states to (vector) outputs, which are often viewed as a distribution over the possible labels

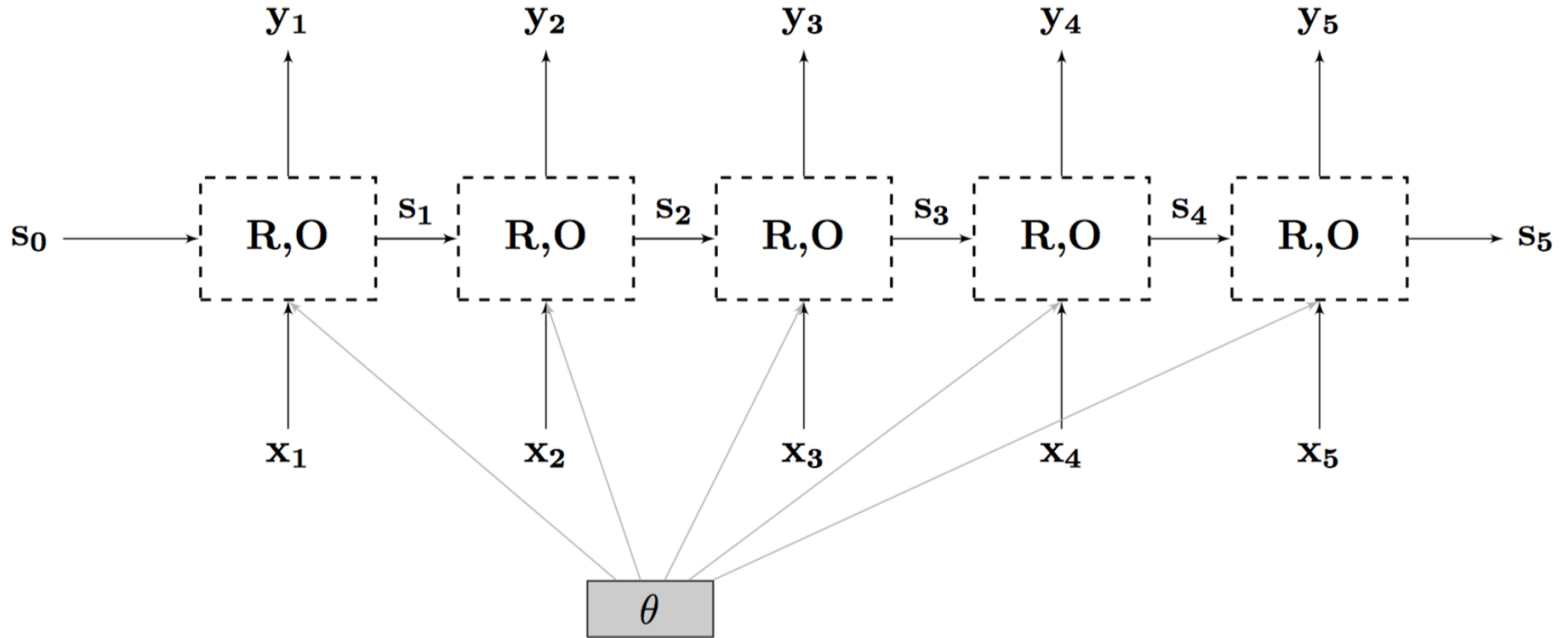
- Example:

$$R_{Elman}(s, x) = \tanh(W[x, s] + b)$$

$$O(s_i) = \text{softmax}(W's_i + b')$$

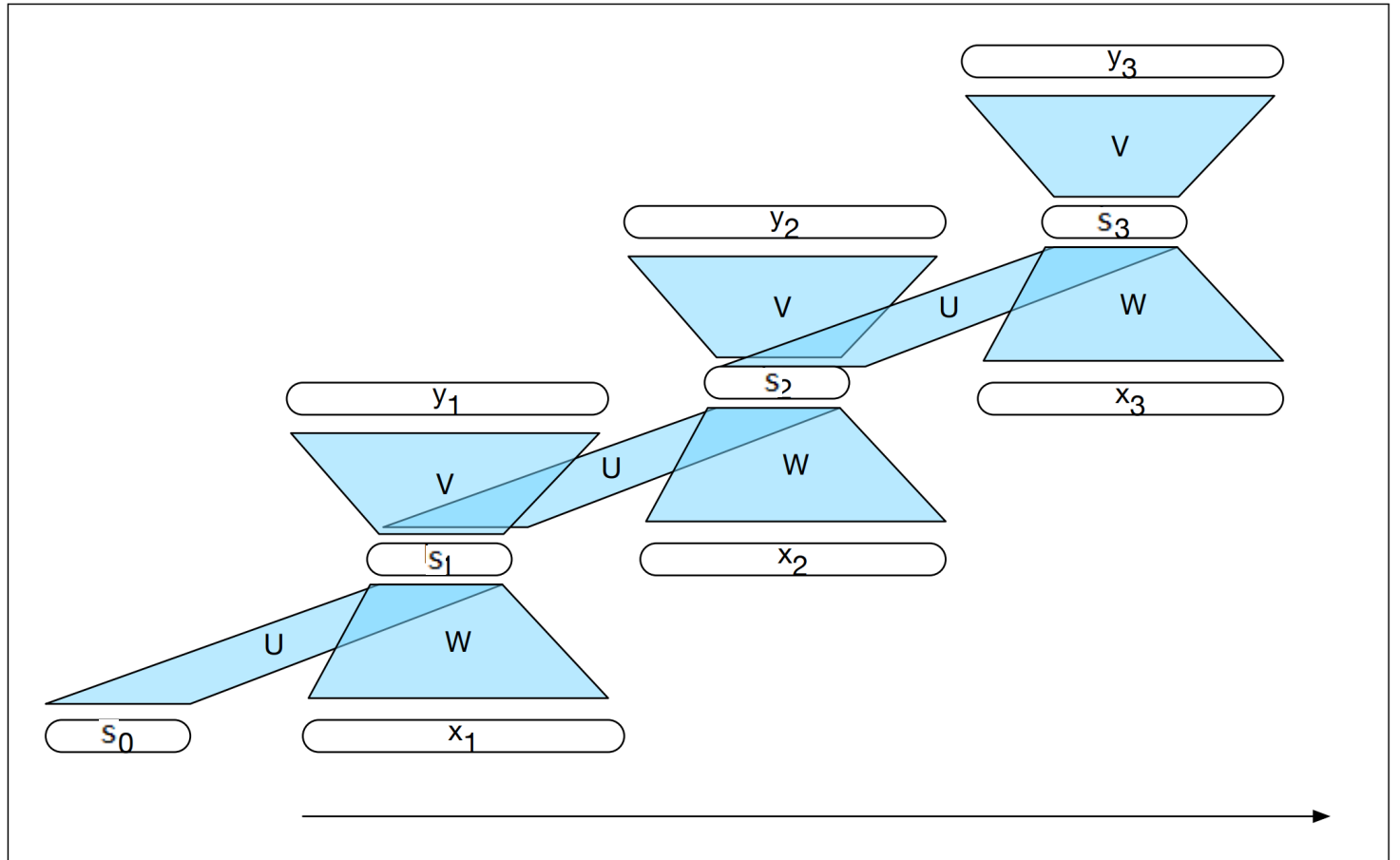
- Conceptually:
 - R is the function embedding the history in a vector
 - O is the function relating the embedded history and the output
 - They are learned jointly

RNNs: Graphical Representation



Recurrent Neural Language Models

- An “unrolled” view of an RNN
- The network stays the same at each time stamp. What’s different are the inputs and previous hidden states



Reminder: Loss Functions

- Training is done by defining a loss function (i.e. a function that determines how “bad” a classification is relative to the gold standard)
- The network then attempts to minimize the empirical risk. For a sample $S=\{x_1, \dots, x_m\}$ with gold standard labels $\{y_1, \dots, y_m\}$, the empirical risk is:

$$L_S(\theta) = \frac{1}{m} \sum_{i=1}^m \ell(\theta; (x_i, y_i))$$

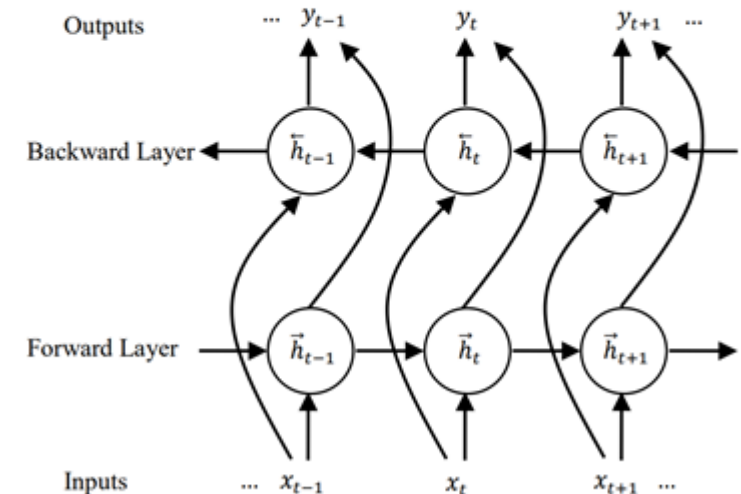
Back to Sentiment Analysis

- When using RNNs for sentence classification (such as sentiment analysis), it is often practical to use Bi-RNNs
- Bi-RNNs:
 - 2 RNNs, one going back to forth and the other forth to back
 - Output function is a function of both states
- This allows the states associated with each word to encode relevant information from the words following them and preceding them

$$\vec{h}_t = R(W_{x\vec{h}}x_t + W_{\vec{h}\vec{h}}\vec{h}_{t-1} + b_{\vec{h}}) \quad (9)$$

$$\overleftarrow{h}_t = R(W_{x\overleftarrow{h}}x_t + W_{\overleftarrow{h}\overleftarrow{h}}\overleftarrow{h}_{t+1} + b_{\overleftarrow{h}}) \quad (10)$$

$$y_t = W_{\vec{h}y}\vec{h}_t + W_{\overleftarrow{h}y}\overleftarrow{h}_t + b_y \quad (11)$$



Back to Sentiment Analysis

- One simple way to do sentiment analysis (or other sentence classification) with Bi-RNNs is to average the output sequence:

$$y = \frac{1}{N} \sum_i y_i$$

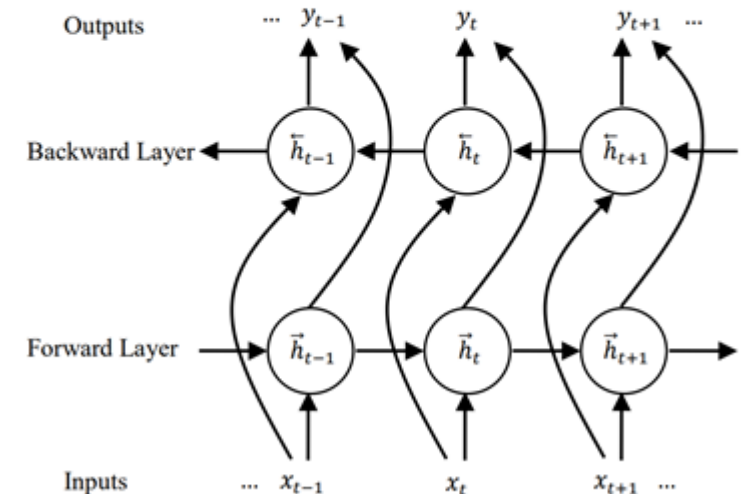
- Now train a binary (log-linear) classifier for predicting the sentiment:

$$P(+|y) = \frac{1}{1+e^{-(w^t \cdot y + b)}} = \text{sigmoid}(w^t \cdot y + b)$$

$$\vec{h}_t = R(W_{x\vec{h}}x_t + W_{\vec{h}\vec{h}}\vec{h}_{t-1} + b_{\vec{h}}) \quad (9)$$

$$\overleftarrow{h}_t = R(W_{x\overleftarrow{h}}x_t + W_{\overleftarrow{h}\overleftarrow{h}}\overleftarrow{h}_{t+1} + b_{\overleftarrow{h}}) \quad (10)$$

$$y_t = W_{\vec{h}y}\vec{h}_t + W_{\overleftarrow{h}y}\overleftarrow{h}_t + b_y \quad (11)$$

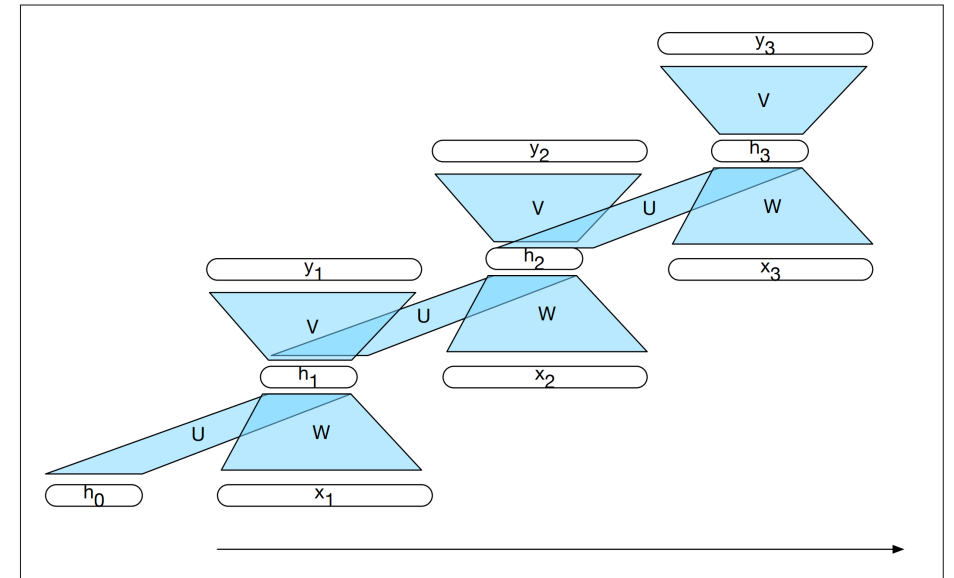


Context in RNN models for Sentiment Analysis

- RNN models are able to take the context (both preceding and following) into account, as well as the linear order between the words
 - bag-of-words models cannot
- They indeed show better performance in tasks such as sentiment analysis
- However, the state sequence is not easy to interpret
 - Much heavier computationally, especially training
- Further investigation is needed to establish what contextual and semantic aspects of sentences are captured using these techniques

A word on Backpropagation Through Time

- In order to train the network, we need to decide what output of the network we would like to compare to the gold standard and what loss function to use
- Even if the recurrent function R and the output function O are shallow (e.g., a single affine function), the computation graph may be deep due to the recurrence
- It is common practice to limit (truncate) the number of iterations
- We will see an example later



Generative RNNs

Decoder Models and Language models

Neural Language Models (LMs)

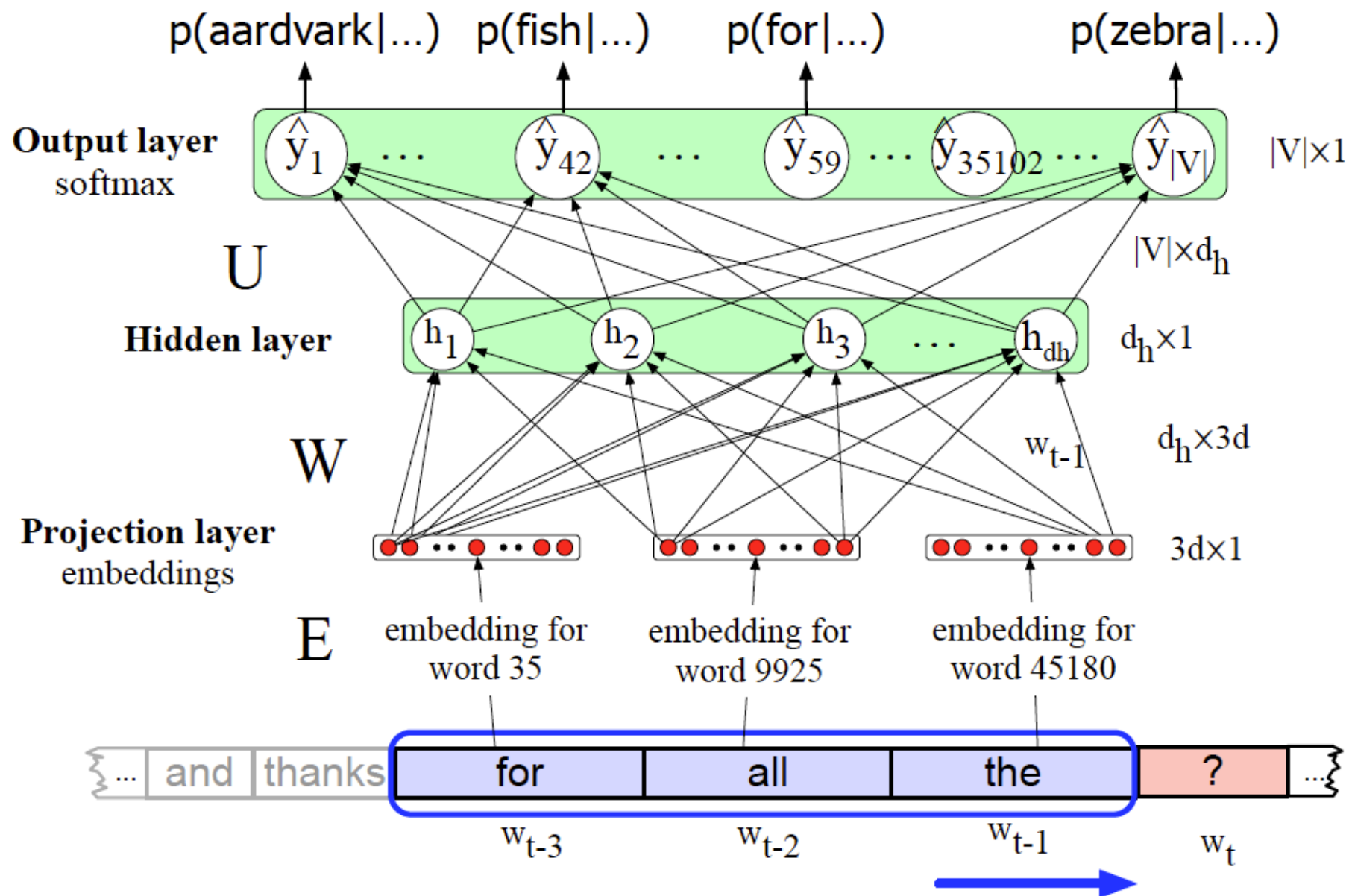
- Recall language modeling
- We've seen n-gram based LMs
- Neural network LMs far outperform n-gram language models
- State-of-the-art neural LMs are based on more powerful neural network technology like Transformers
 - We will talk about them later
- But **simple feedforward LMs** can do well too

Simple FF Neural Language Models

- **Task:** predict next word w_t
- given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \dots$
- **Problem:** Now we're dealing with sequences of arbitrary length
- **Solution:** Sliding windows (of fixed length)

$$P(w_t | w_{t-1}, \dots, w_1) \approx P(w_t | w_{t-1}, \dots, w_{t-N+1})$$

Neural Language Model



Generalization in Neural Language Models

- **Training data:**

- We've seen: I have to make sure that the cat gets fed.
- Never seen: dog gets fed

- **Test data:**

- I forgot to make sure that the dog gets ____
- n-gram LM can't predict "fed"!
- Neural LM will implicitly learn the similarity of "cat" and "dog" embeddings to generalize and predict "fed" after dog

Recurrent Neural LMs

output distribution

$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{U}\mathbf{h}^{(t)} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden states

$$\mathbf{h}^{(t)} = \sigma(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_e \mathbf{e}^{(t)} + \mathbf{b}_1)$$

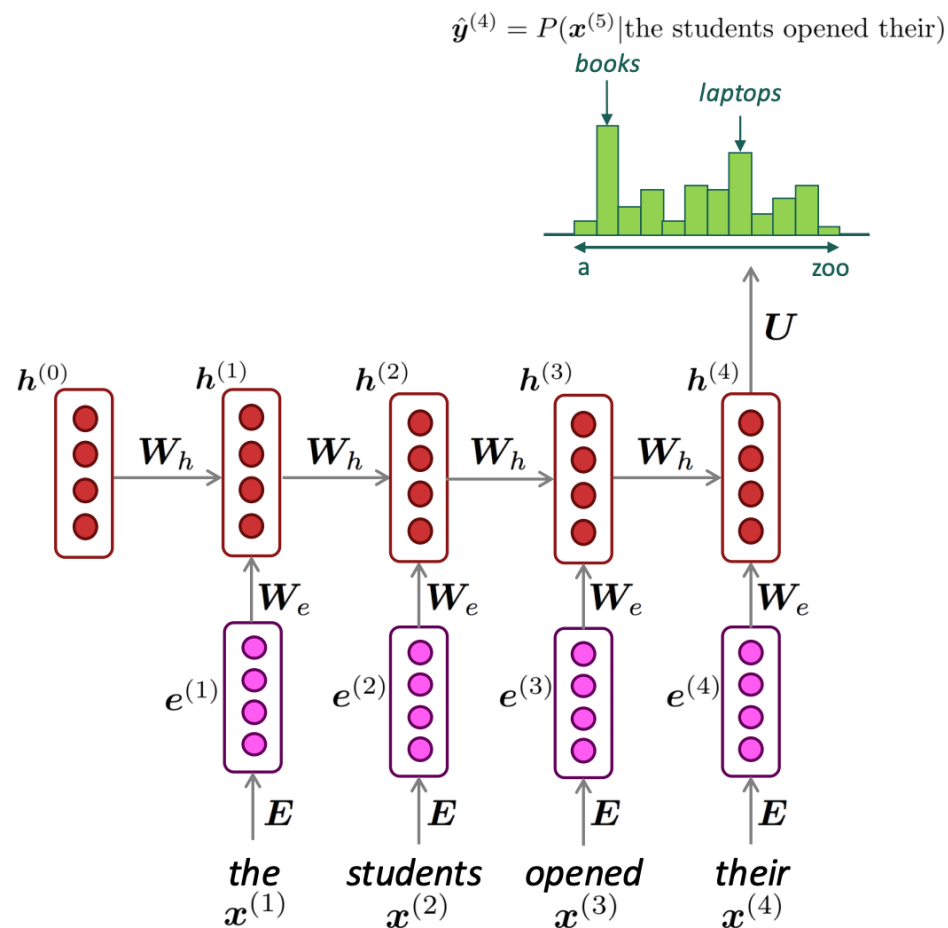
$\mathbf{h}^{(0)}$ is the initial hidden state

word embeddings

$$\mathbf{e}^{(t)} = \mathbf{E}\mathbf{x}^{(t)}$$

words / one-hot vectors

$$\mathbf{x}^{(t)} \in \mathbb{R}^{|V|}$$



Recurrent Neural Language Models

- Language models based on RNNs have shown much power in the past decade years, consistently surpassing n-gram models
- The basic architecture is that of a sequence recurrent neural net (RNN)
- Just like in Feed Forward LMs:
 - Input words are converted into embeddings
 - The output is passed through a softmax layer, which defines the probability of the next word
- The difference is that the hidden state is passed as input to the new layer

Reminder: Loss Functions

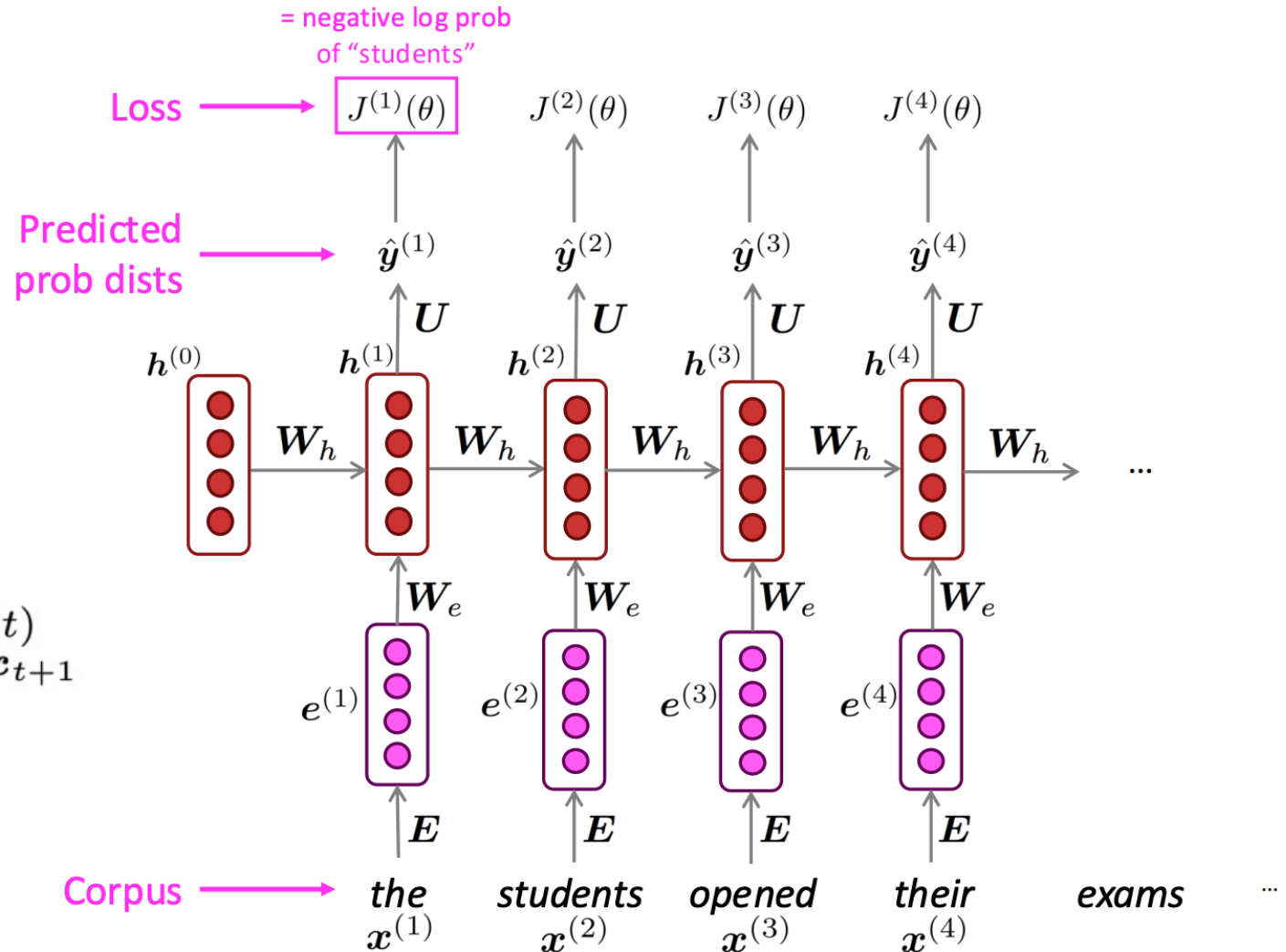
- Training is done by defining a loss function (i.e. a function that determines how “bad” a classification is relative to the gold standard)
- The network then attempts to minimize the empirical risk. For a sample $S=\{x_1, \dots, x_m\}$ with gold standard labels $\{y_1, \dots, y_m\}$, the empirical risk is:

$$\frac{1}{m} \sum_{i=1}^m \ell(\theta; (x_i, y_i))$$

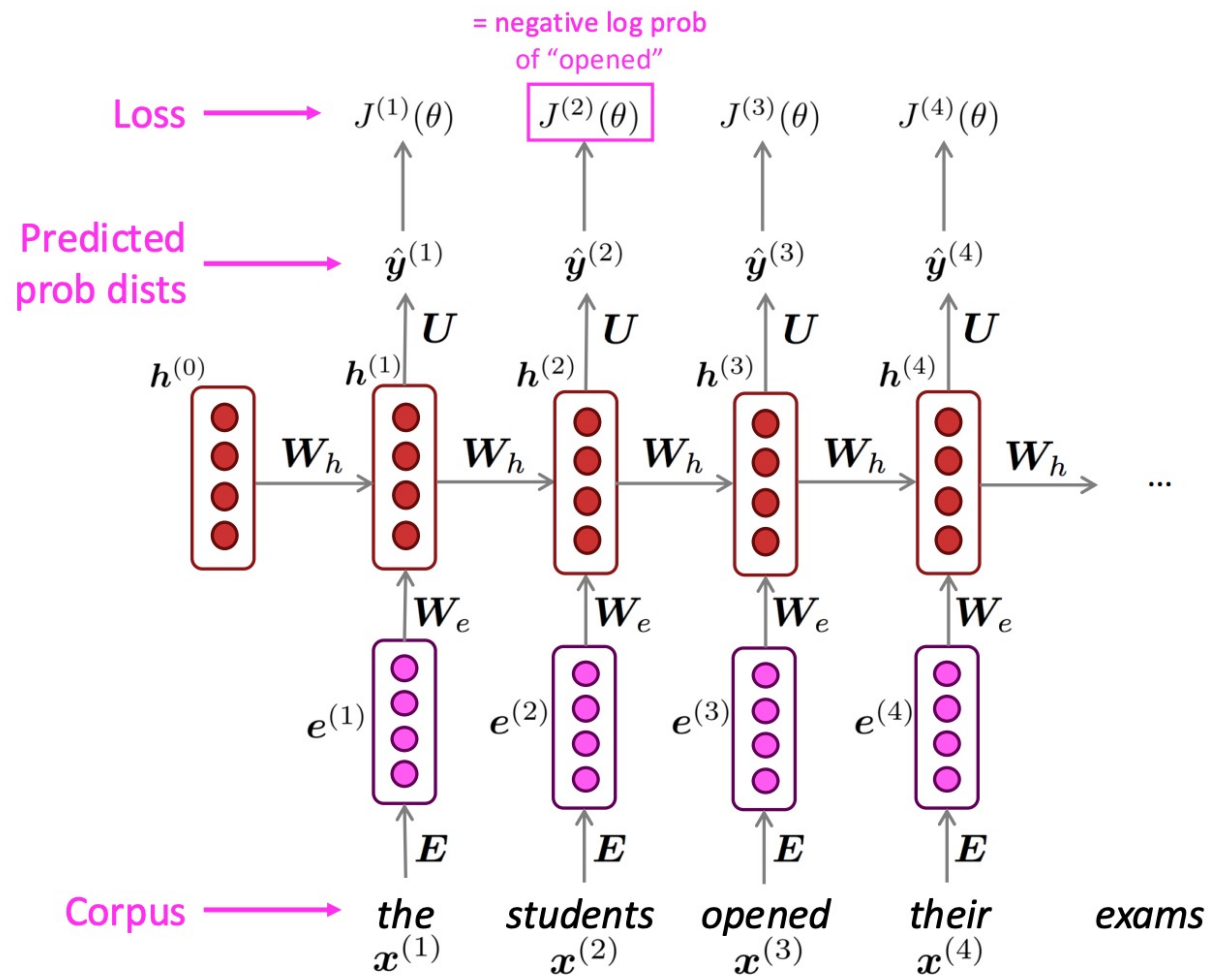
Training RNN LMs

- Neural language models are generally trained to maximize the sum of the predicted log-probabilities of the correct words
 - Cross-entropy loss

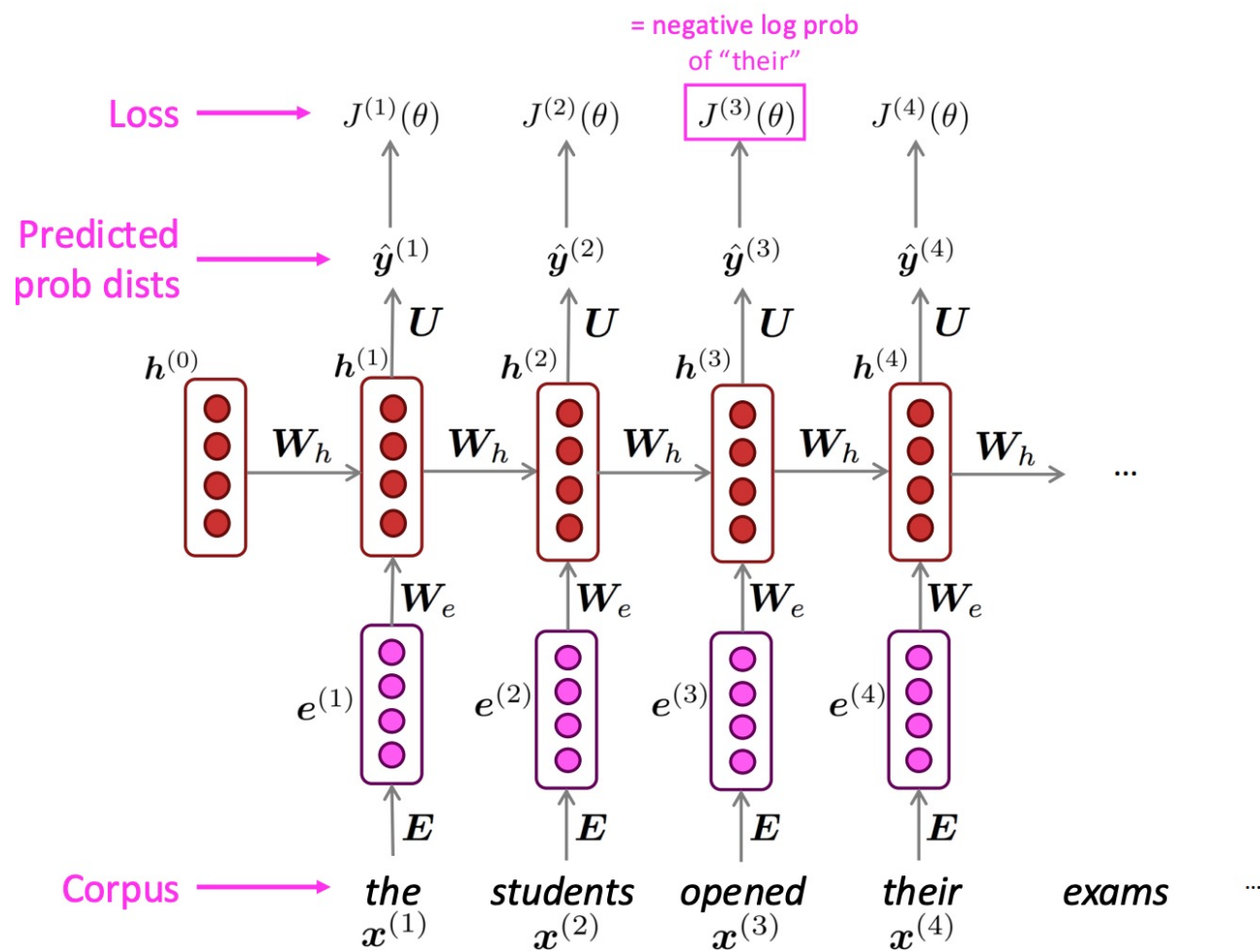
$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T -\log \hat{y}_{x_{t+1}}^{(t)}$$



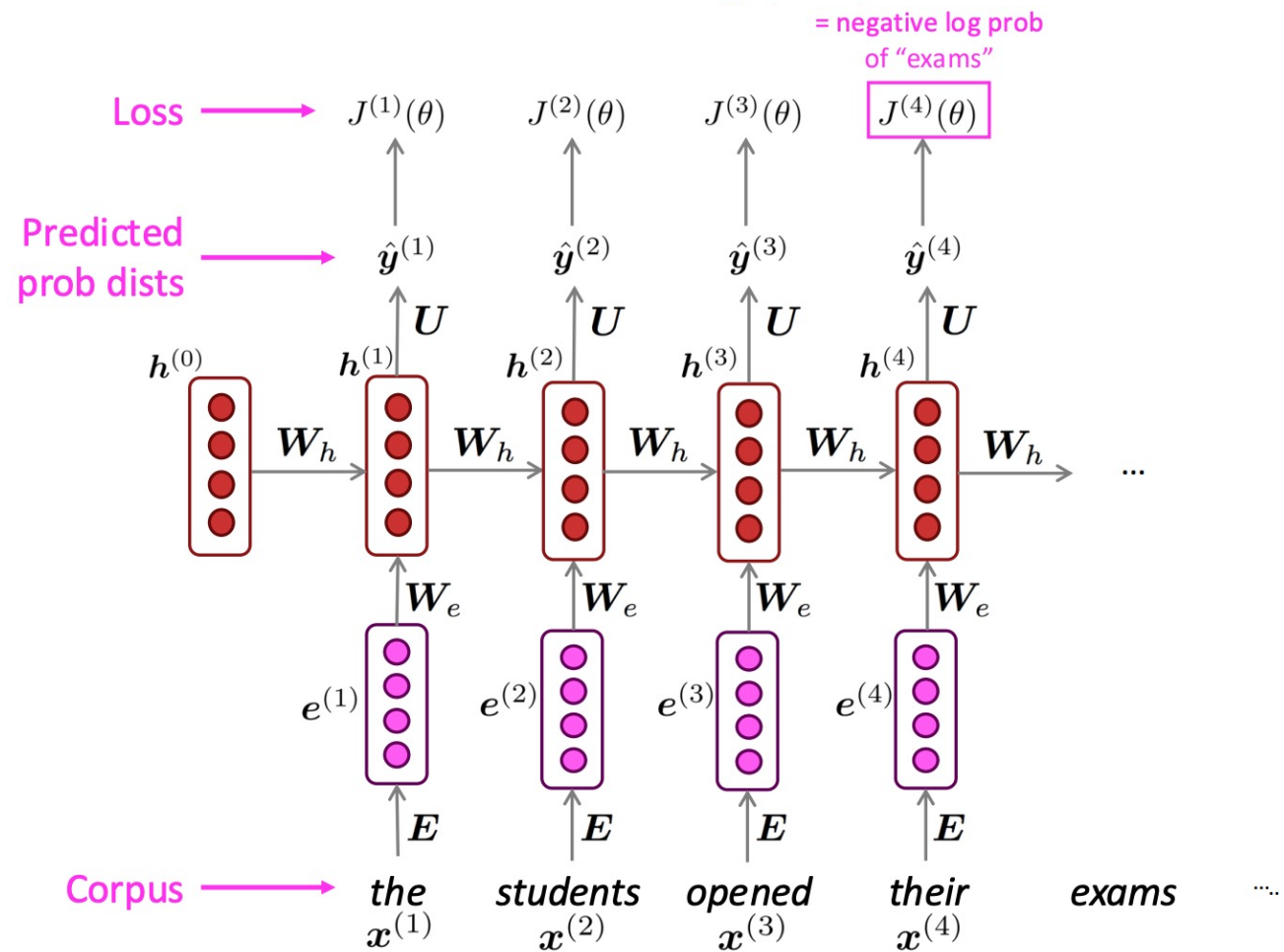
Training RNN LMs



Training RNN LMs



Training RNN LMs



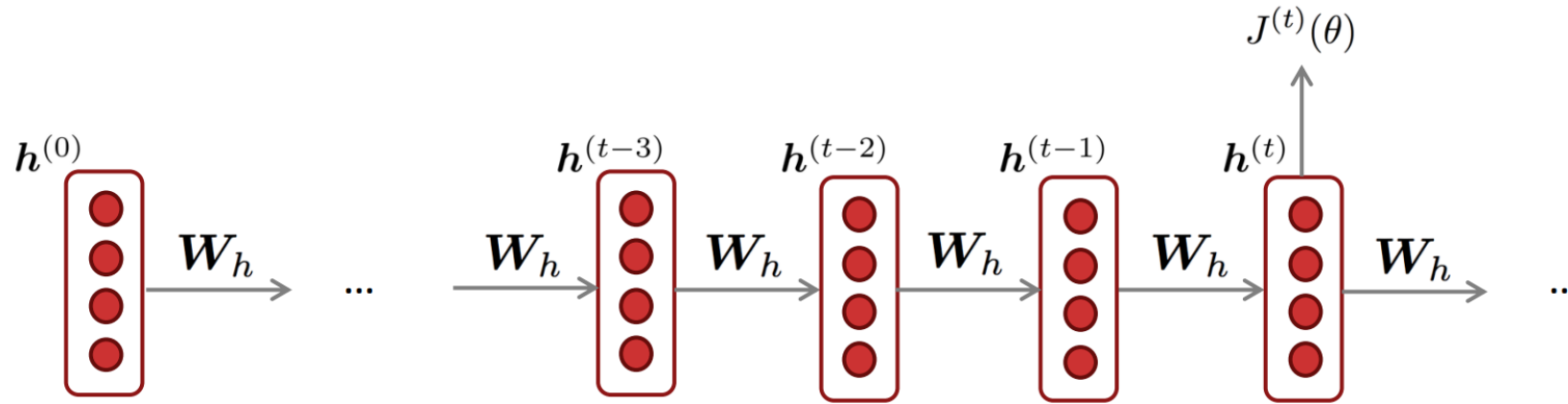
Training RNN Language Model

- However:
 - Computing loss and gradients across entire corpus at once is too expensive (memory-wise)!

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider as a sentence (or a document)

Backpropagation for RNNs

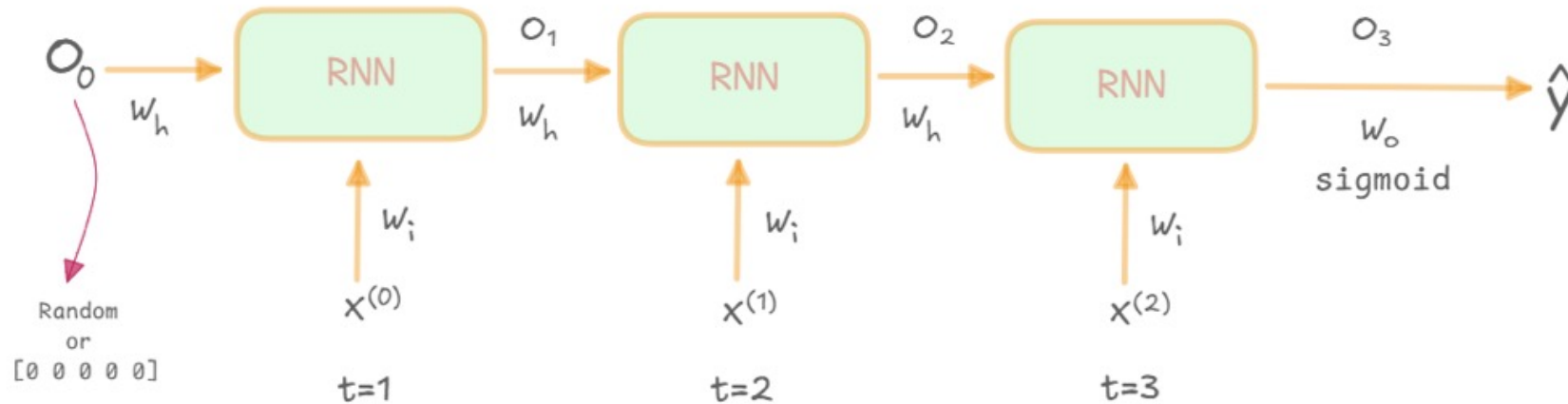


Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t. the **repeated** weight matrix W_h ?

Answer:
$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$$

“The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears”

Backpropagation for RNNs: Simple Case



$$O_1 = f \left(X_i^{(0)} W_i + O_0 W_h \right)$$

$$O_2 = f \left(X_i^{(1)} W_i + O_1 W_h \right)$$

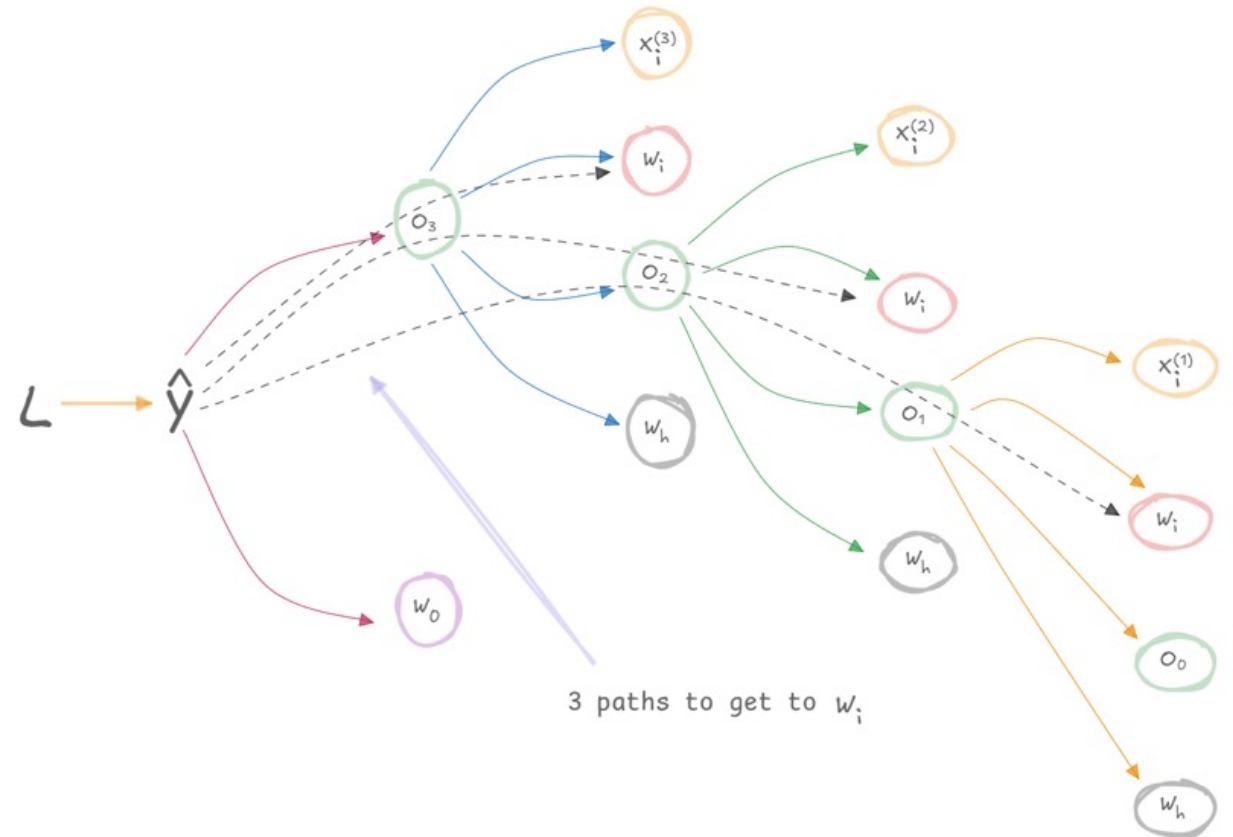
$$O_3 = f \left(X_i^{(2)} W_i + O_2 W_h \right)$$

$$\hat{y} = \sigma (O_3 W_o)$$

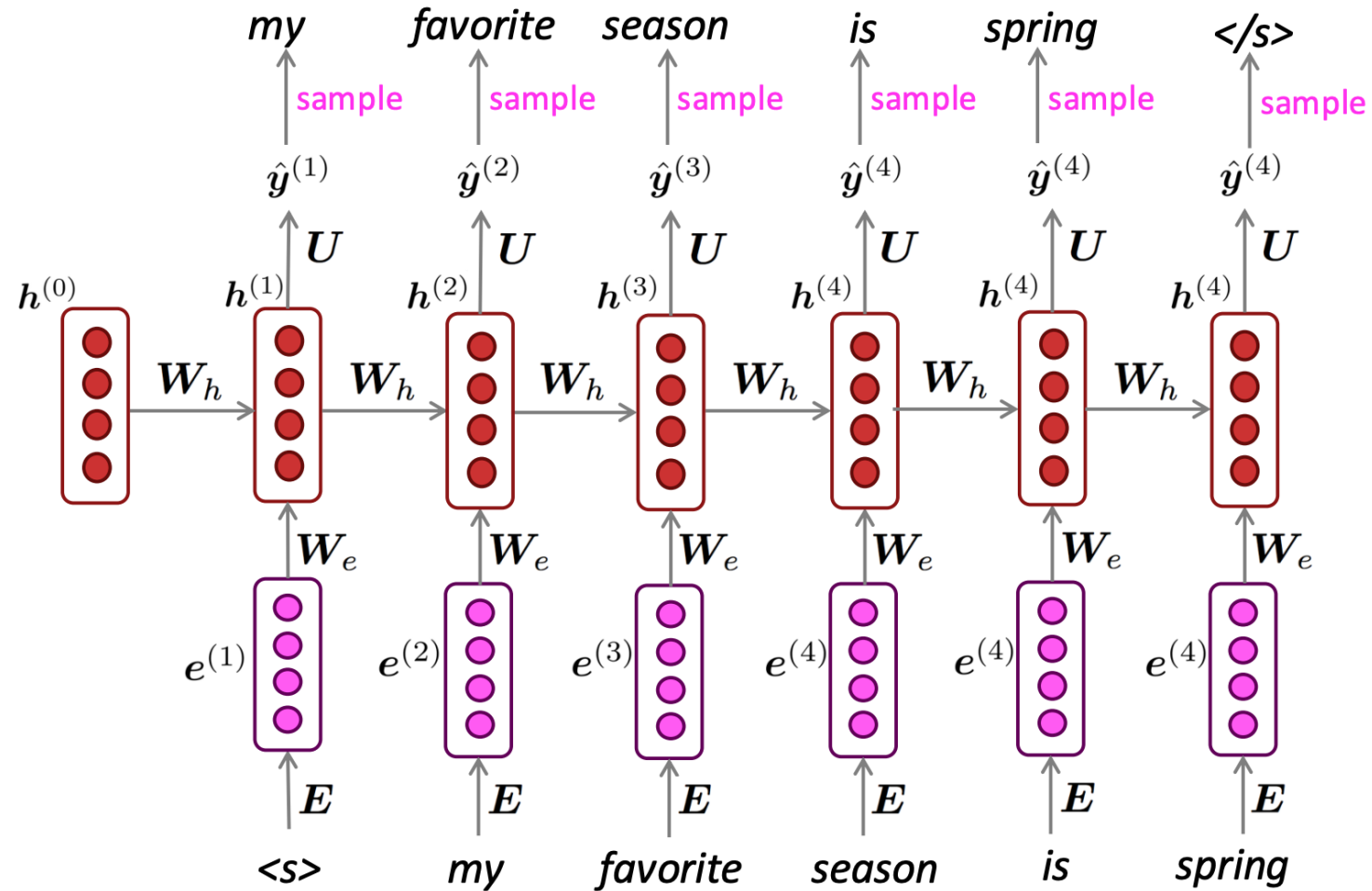
Backpropagation for RNNs: Simple Case

- Assume L is the loss

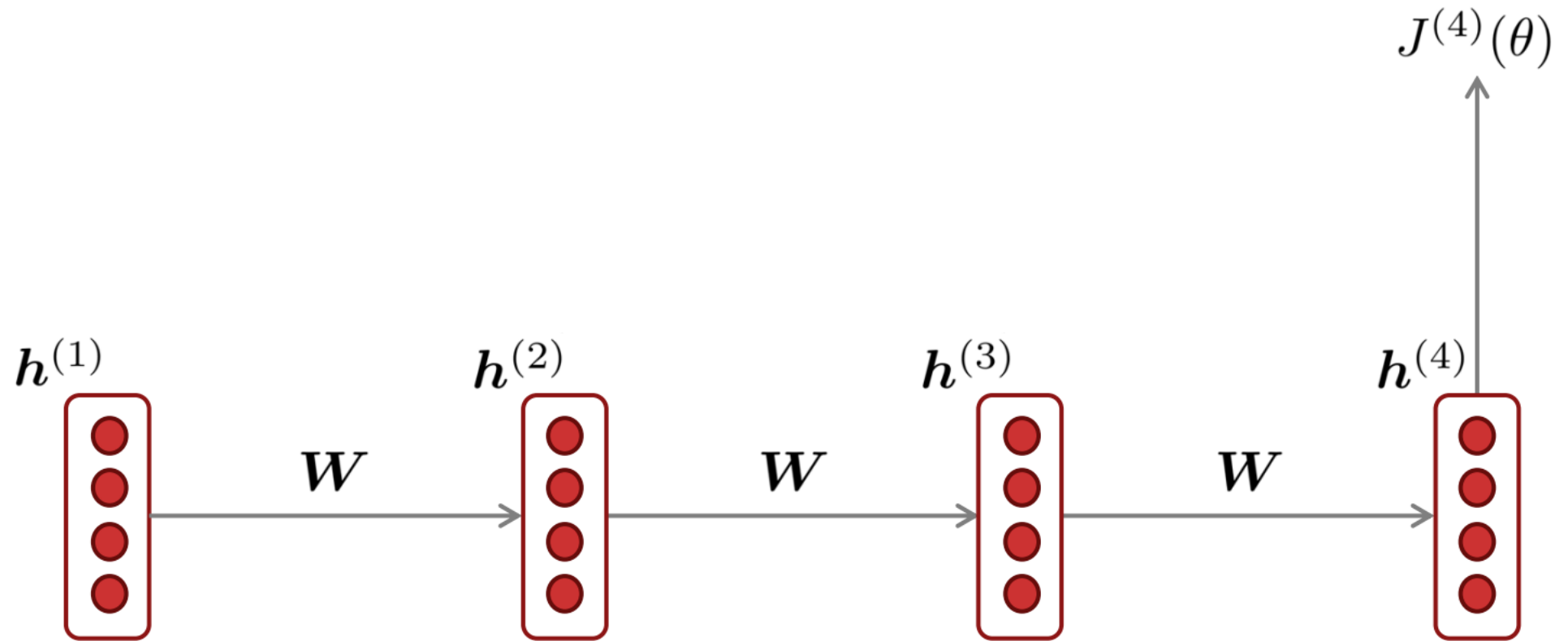
$$\begin{aligned} \frac{d\mathcal{L}}{dW_h} &= \frac{d\mathcal{L}}{d\hat{y}} \cdot \frac{d\hat{y}}{dO_3} \cdot \frac{dO_3}{dW_h} \\ &+ \frac{d\mathcal{L}}{d\hat{y}} \cdot \frac{d\hat{y}}{dO_3} \cdot \frac{dO_3}{dO_2} \cdot \frac{dO_2}{dW_h} \\ &+ \frac{d\mathcal{L}}{d\hat{y}} \cdot \frac{d\hat{y}}{dO_3} \cdot \frac{dO_3}{dO_2} \cdot \frac{dO_2}{dO_1} \cdot \frac{dO_1}{dW_h} \end{aligned}$$



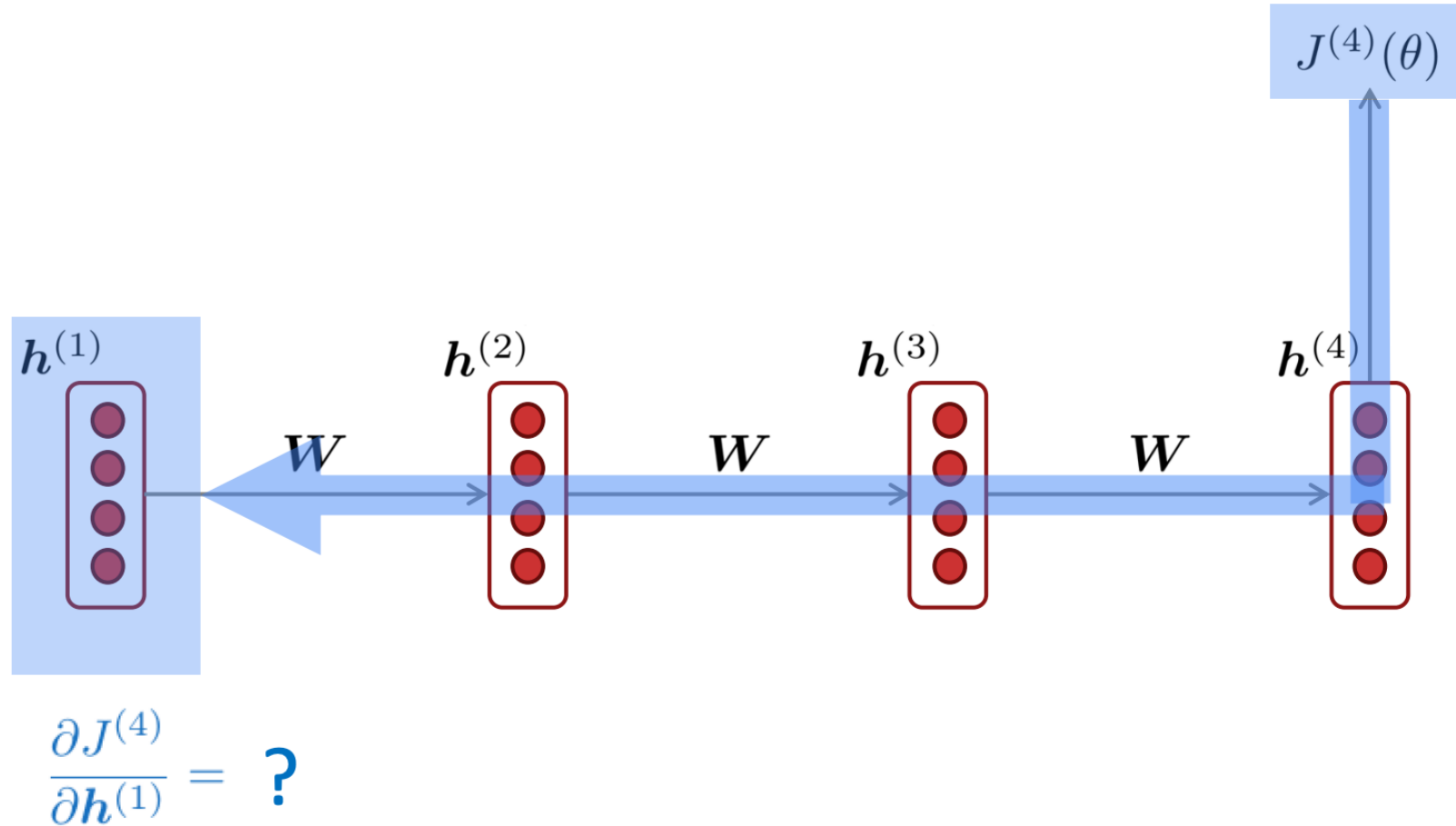
Inference with RNN LM



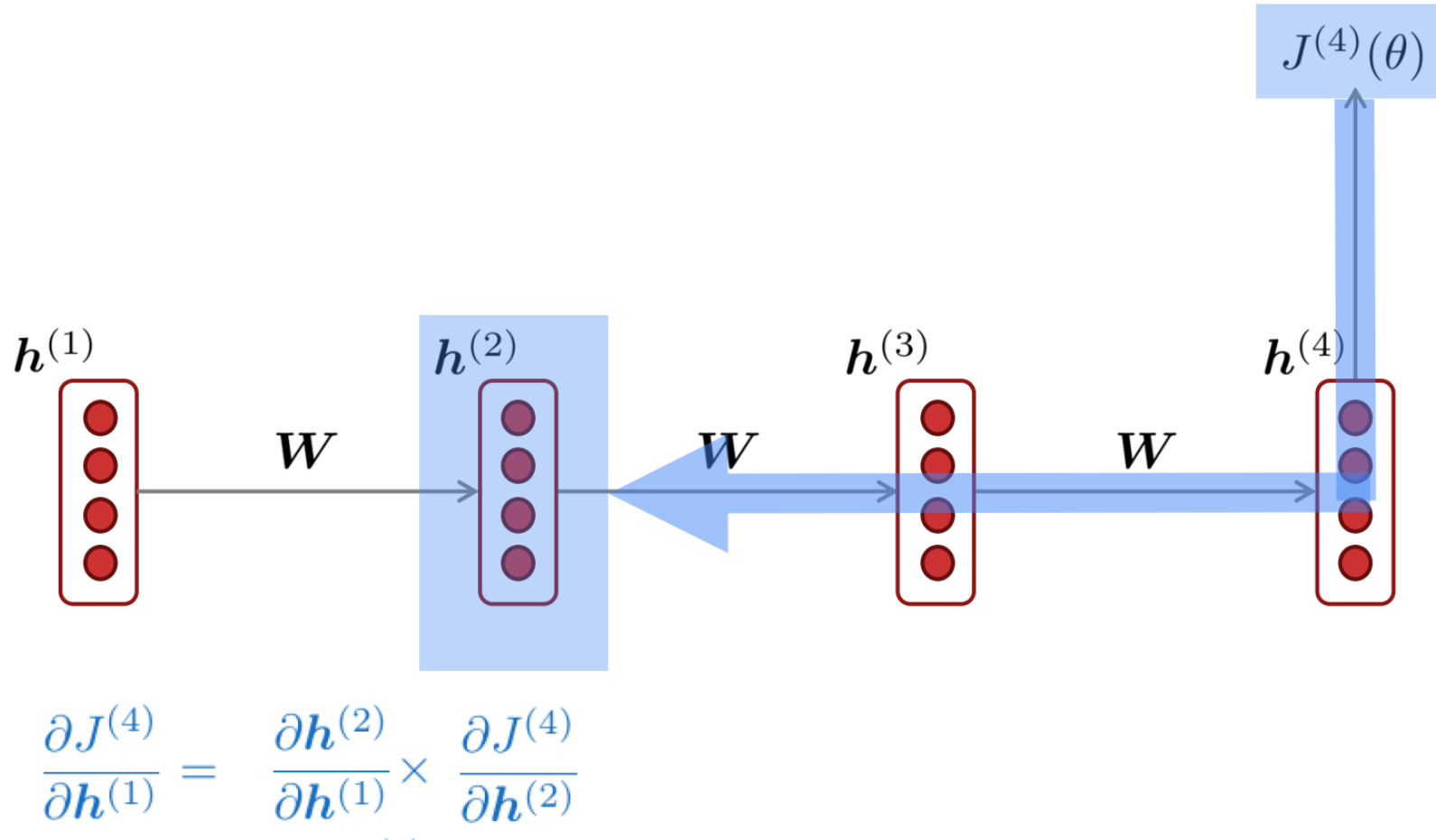
Vanishing and Exploding Gradients: Intuition

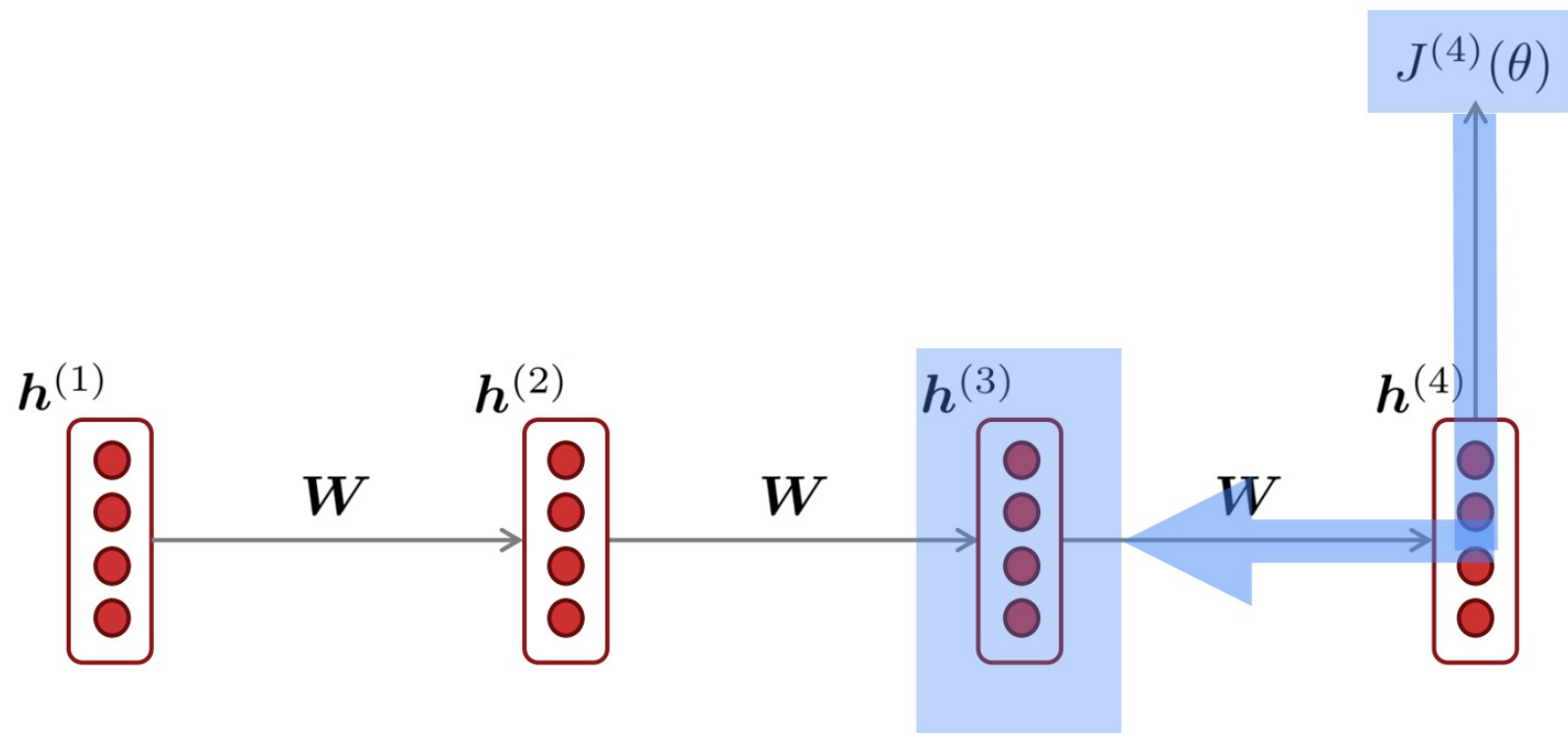


Vanishing and Exploding Gradients: Intuition



Vanishing and Exploding Gradients: Intuition

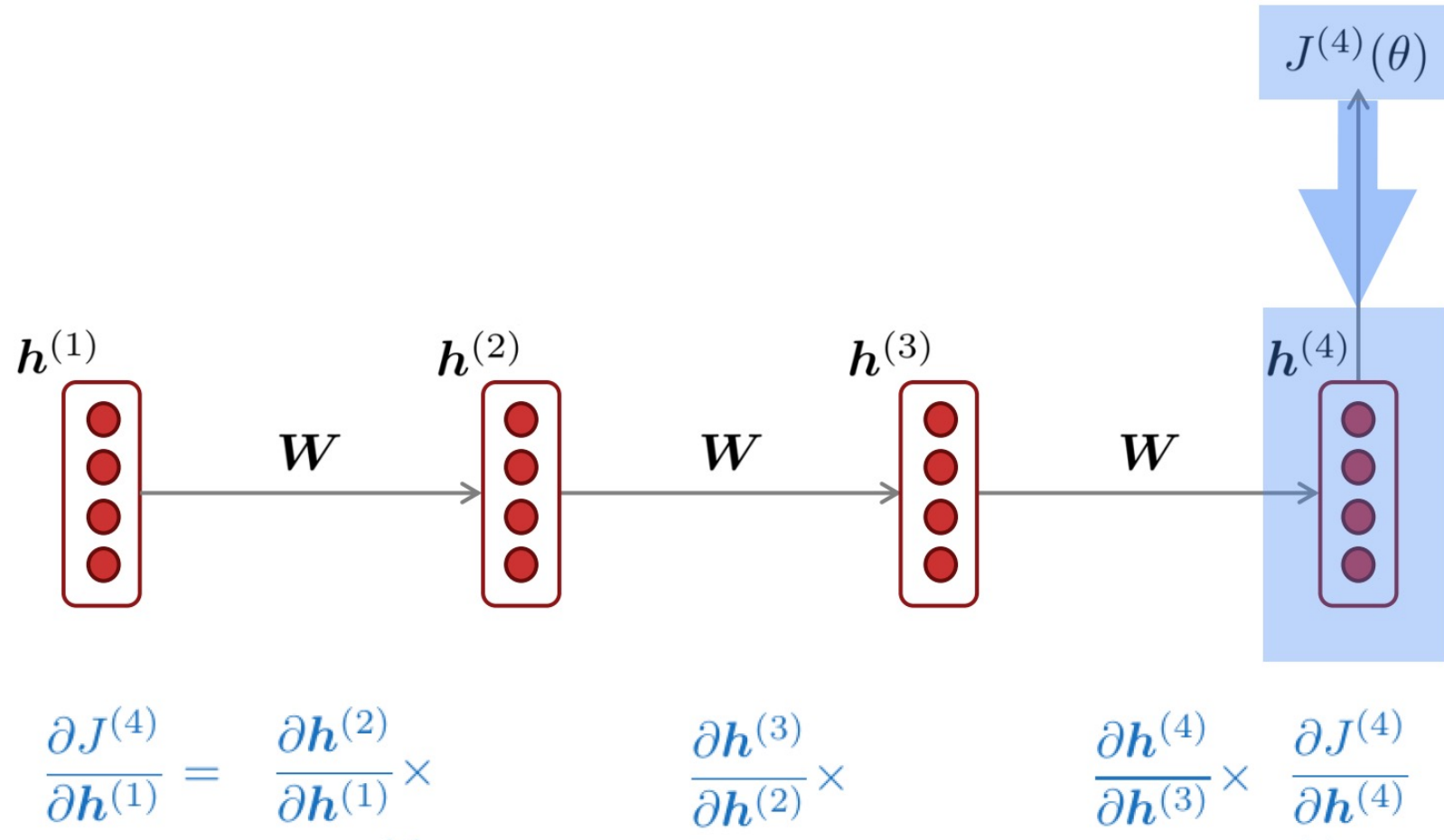


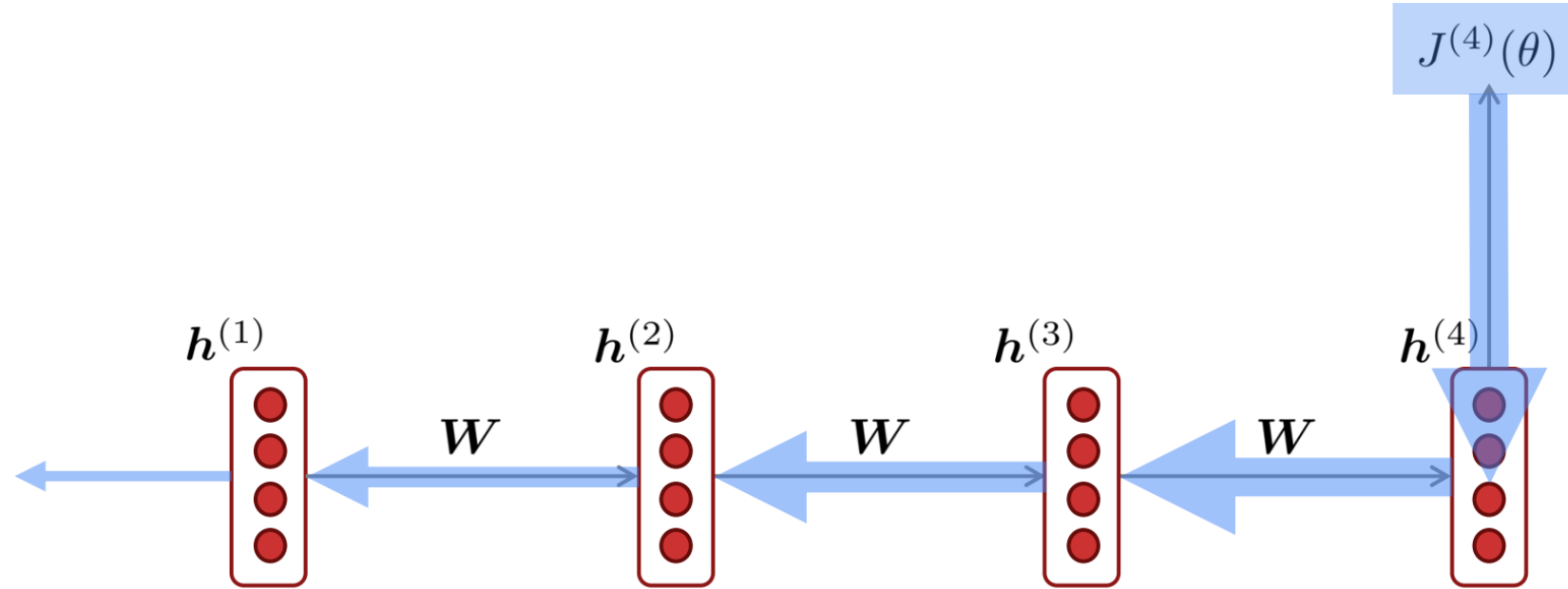


$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times$$

$$\frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial J^{(4)}}{\partial h^{(3)}}$$

Vanishing and Exploding Gradients: Intuition





$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \left[\frac{\partial h^{(2)}}{\partial h^{(1)}} \right] \times \left[\frac{\partial h^{(3)}}{\partial h^{(2)}} \right] \times \left[\frac{\partial h^{(4)}}{\partial h^{(3)}} \right] \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Vanishing Gradient Problem

- The difficulty with vanishing gradients is that it makes it very difficult for the model to learn long-distance dependencies
- The effect of changes to the state in the n -th position before, is miniscule
 - Therefore, the hidden state in the n -th position before has no causal effect on the prediction n positions ahead
- This was addressed by a different RNN architecture called LSTM (Long Short Term Memory; Hochreiter and Schmidhuber, 1997)

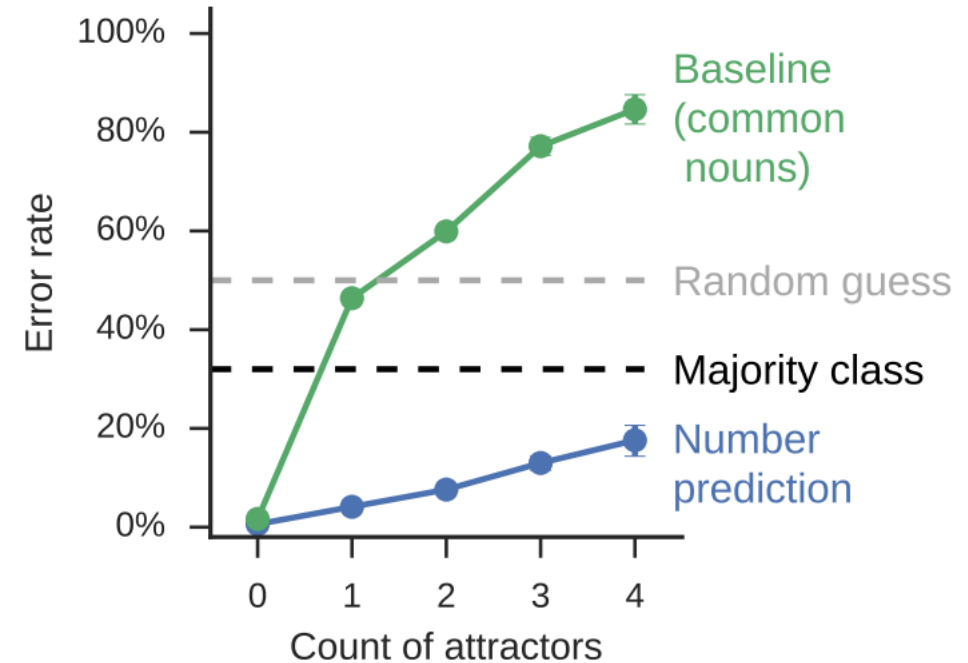
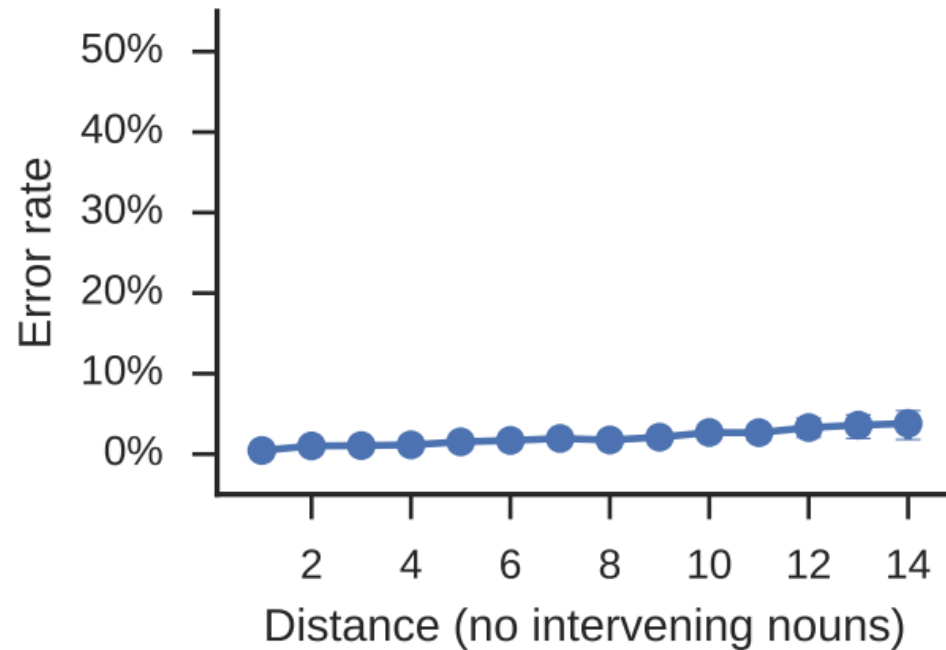
Quantifying LSTM's Ability to Capture Long-distance Dependencies

- Lizen et al. (2016) cast the problem as a subject-verb agreement problem
 - They tested mostly on English
 - They measured the effect both of distance and distractors on the error rate

(9) The **roses** in the vase by the door **are** red.

(10) The **roses** in the vase by the *chairs* **are** red.

Quantifying LSTM's Ability to Capture Long-distance Dependencies



The baseline is only exposed to the identity of the nouns, without the syntactic structure